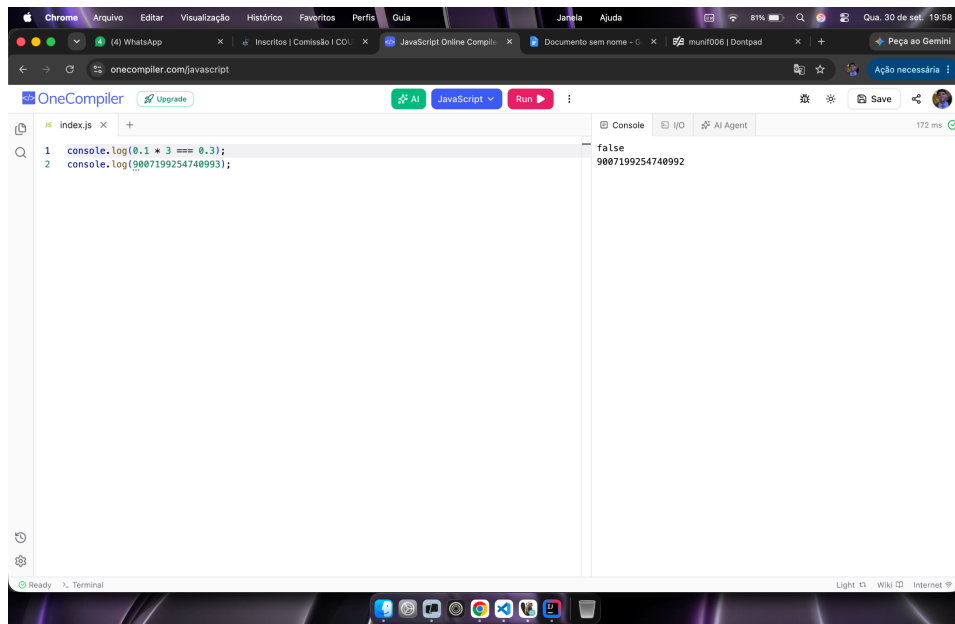


1 - JAVASCRIPT



ENTRADA: `0.1 * 3 === 0.3`

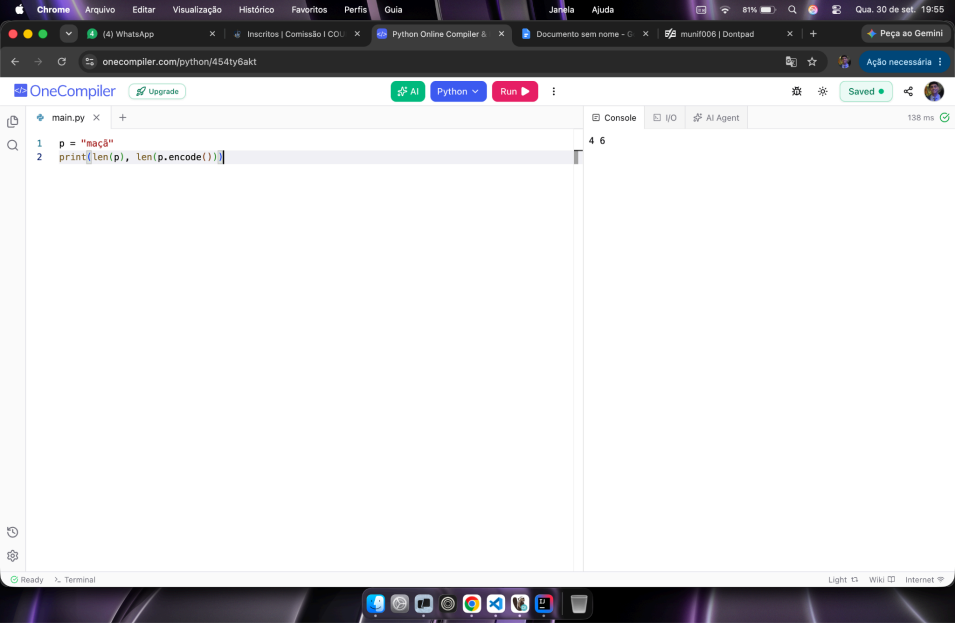
SAIDA: `false`

ENTRADA: `9007199254740993`

SAIDA: `9007199254740992`

Resposta b) : Nunca é detectado. Nenhum erro acontece. O `0.1 * 3 === 0.3` dá `false` por causa da imprecisão do ponto flutuante (`0.30000000000000004`), e `9007199254740993` é impresso como `9007199254740992`, porque excede o maior inteiro seguro (2^{53}). É um comportamento silencioso, sem erro nem aviso.

2 - PYTHON



The screenshot shows a web browser window with the OneCompiler Python IDE. The code editor contains the following Python code:

```
1 p = "maçã"
2 print(len(p), len(p.encode()))
```

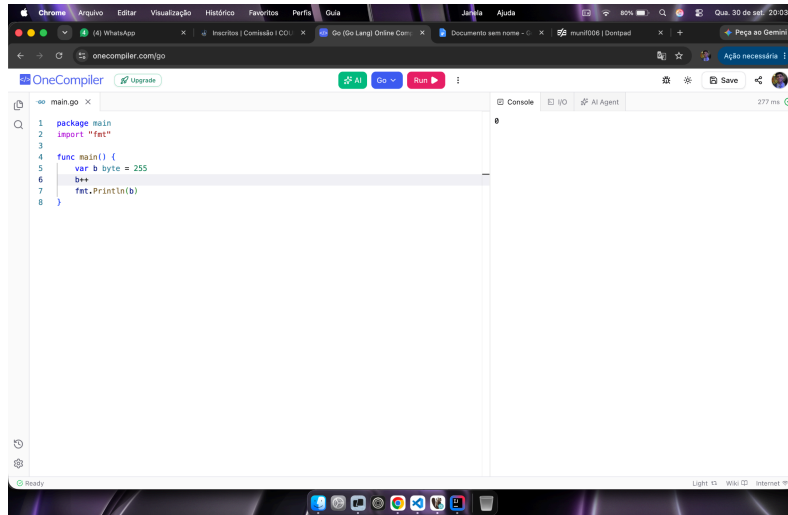
The output console on the right displays the result of the execution: 4 6. The first value, 4, represents the number of characters in the string "maçã", and the second value, 6, represents the number of bytes in the UTF-8 encoded string, where the 'ç' character is encoded as two bytes.

ENTRADA: `p = "maçã"`

SAIDA: 4 6

Resposta b) : Nunca. Não há problema nenhum, o código é válido. A saída é 4 5, já que `len(p)` conta caracteres e `len(p.encode())` conta bytes em UTF-8 (o "ç" ocupa 2 bytes).

3 - GO

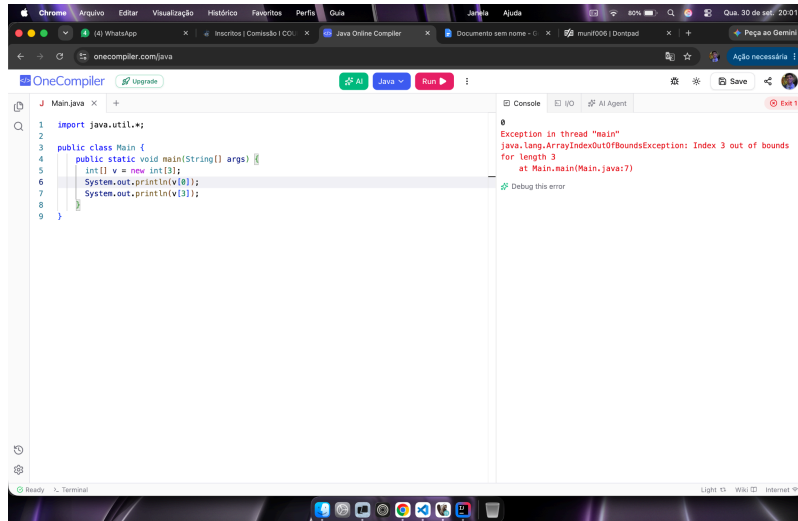


Entrada: `var b byte = 255`

Saída: 0

Resposta b) : Nunca. A constante 255 cabe num byte, então compila normalmente. Já o `b++` causa overflow silencioso (wraparound) em tempo de execução, sem pânico, e a saída é 0.

4 - JAVA



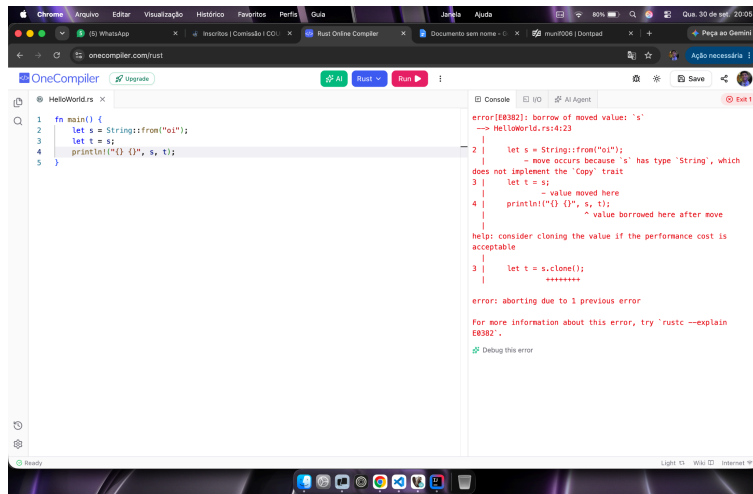
Entrada: `int[] v = new int[3];`

Saida 1: 0

Saida 2: Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for
length 3
at Main.main(Main.java:7)

Resposta b) : Execução. O compilador não verifica índices de array. `v[0]` imprime 0, e `v[3]` lança `ArrayIndexOutOfBoundsException` em tempo de execução.

5 - Rust



Entrada: `let s = String::from("oi");`
`let t = s;`

Saida: `error[E0382]: borrow of moved value: `s``
`--> HelloWorld.rs:4:23`

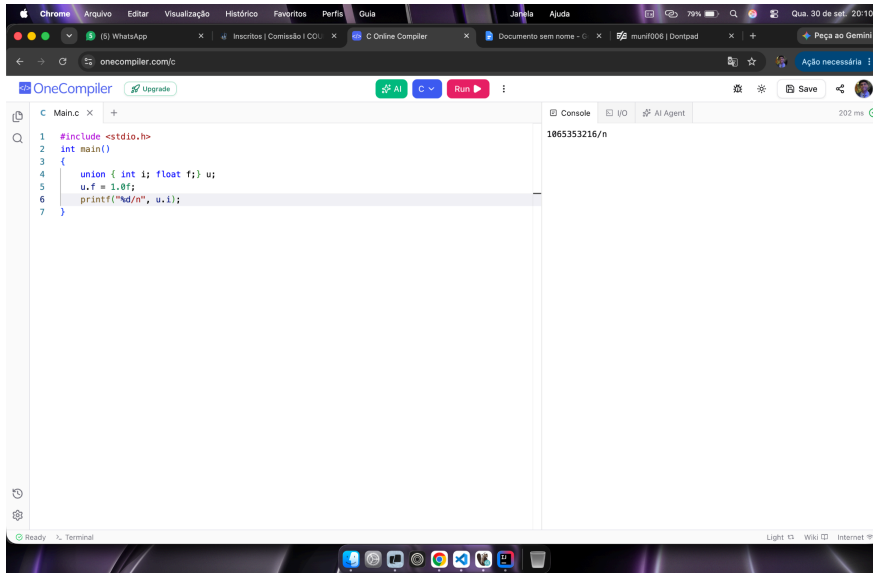
```
|  
2 |     let s = String::from("oi");  
|         - move occurs because `s` has type `String`, which does not  
implement the `Copy` trait  
3 |     let t = s;  
|         - value moved here  
4 |     println!("{}", s, t);  
|                     ^ value borrowed here after move  
|  
help: consider cloning the value if the performance cost is acceptable  
|  
3 |     let t = s.clone();  
|             +++++++
```

`error: aborting due to 1 previous error`

`For more information about this error, try `rustc --explain E0382`.`

Resposta b) : Compilação. `let t = s;` move a posse da String para t, e usar s depois gera o erro de compilação `borrow of moved value: s`. O programa nem chega a ser gerado.

6 - C



```
1 #include <stdio.h>
2 int main()
3 {
4     union { int i; float f; } u;
5     u.f = 1.0f;
6     printf("%d\n", u.i);
7 }
```

Console: 1065353216/n

Entrada: `union { int i; float f; } u;`
`u.f = 1.0f;`

Saida:1065353216/n

Resposta b) : Nunca (como erro). Compila e executa sem erro. Ler u.i depois de escrever u.f reinterpreta os bits do float; em C isso é permitido (type punning via union). A saída é 1065353216 (0x3F800000, a representação IEEE 754 de 1.0f).